
第 3 章 python 进阶

在第 3 章中，我们学习了 Python 的基础语法，掌握了如何编写脚本来处理简单的任务。但在面对复杂的 AI 系统（如设计一个深度学习框架、管理庞大的数据集或封装一个训练引擎）时，零散的函数和变量会变得难以维护。

本节将深入探讨 Python 中类与对象的底层机制、封装继承多态的实现方式，以及魔法方法和生成器在 AI 数据处理中的实际应用。

3.1 面向对象编程 (OOP) 进阶

面向对象不仅仅是一种语法，更是一种思维方式。它将数据和操作数据的方法“封装”在一起，形成“对象”。在 AI 开发中，万物皆对象：一个模型是一个对象，一个数据集是一个对象，甚至一个优化器也是一个对象。

3.1.1 类与对象的核心机制回顾

1. 类 (Class) 与 实例 (Instance)

类：是一种用户自定义的数据类型。它定义了一组属性（变量）和方法（函数），描述了该类型对象的共同特征和行为。

实例：是根据类创建的具体对象。每个实例在内存中拥有独立的存储空间，用于保存其属性值。

2. self 参数的本质

在类的方法定义中，第一个参数通常命名为 `self`。它是一个指向调用该方法的实例本身的引用。Python 在调用实例方法时，会自动将该实例作为第一个参数传入。这确保了方法能够访问和修改属于该特定实例的属性。

```
class AIModel:
```

```
    # __init__ 是初始化方法，在实例化时自动调用，用于初始化实例属性
```

```
    def __init__(self, name, architecture):
```

```
        self.name = name           # 实例属性：模型名称
```

```
        self.architecture = architecture # 实例属性：模型架构
```

```
        self.is_trained = False    # 实例属性：训练状态
```

```

def train(self):
    # 通过 self 访问和修改实例属性
    print(f'开始训练模型: {self.name}')
    self.is_trained = True

# 实例化：创建两个独立的对象
model_a = AIModel("Model_A", "CNN")
model_b = AIModel("Model_B", "Transformer")

# 方法调用
model_a.train()

# 输出：开始训练模型: Model_A

# 说明：调用时无需传递 self，Python 解释器会自动处理

print(f'Model A 训练状态: {model_a.is_trained}') # 输出: True
print(f'Model B 训练状态: {model_b.is_trained}') # 输出: False (不同实例互不影响)

```

3.1.2 封装、继承与多态的应用

1. 封装 (Encapsulation)

封装是指将数据和操作数据的方法绑定在一起，并限制对内部数据的直接访问。

Python 通过属性命名约定来实现访问控制：

- (1) **公有成员 (name)**：可以在类外部直接访问。
- (2) **受保护成员 (_name)**：以单下划线开头。表示该属性仅供内部使用或子类使用，但不强制禁止外部访问（约定俗成）。
- (3) **私有成员 (__name)**：以双下划线开头。Python 解释器会通过名称改写（Name Mangling）机制，将 __name 改写为 _ClassName__name，从而在形式上阻止外部直接访问。

```
class DatabaseConnection:
```

```
def __init__(self, db_name, password):  
    self.db_name = db_name      # 公有属性  
    self.__password = password  # 私有属性
```

```
def connect(self):  
    # 类内部可以访问私有属性  
    if self.__verify(self.__password):  
        print("连接成功")
```

```
def __verify(self, pwd):  
    return len(pwd) > 5
```

```
db = DatabaseConnection("UserDB", "123456")  
db.connect()
```

```
# print(db.__password) # 报错: AttributeError, 无法直接访问私有属性
```

2. 继承 (Inheritance)

继承允许一个类（子类）获取另一个类（父类/基类）的属性和方法。这实现了代码复用和层级关系的构建。在深度学习框架中，自定义模型通常继承自基础模型类（如 `torch.nn.Module`）。

```
class Model: # 父类  
    def save(self):  
        print("执行保存模型逻辑")  
  
class TextModel(Model): # 子类  
    def predict(self, text):  
        print(f"处理文本输入: {text}")  
  
class ImageModel(Model): # 子类
```

```
def predict(self, image):  
    print(f'处理图像输入')
```

```
# 子类实例自动拥有父类的 save 方法  
  
bert = TextModel()  
bert.predict("Data")  
  
bert.save()
```

3. 多态 (Polymorphism)

多态指不同类的对象对同一消息（方法调用）做出不同的响应。在 Python 中，多态依赖于“鸭子类型”（Duck Typing），即只要对象实现了特定的方法，就可以被统一对待，而无需关注其具体类型。

```
def execute_prediction(model_obj, data):  
    # 该函数不关心 model_obj 的具体类型  
    # 只要该对象拥有 predict 方法即可  
    model_obj.predict(data)  
  
models = [TextModel(), ImageModel()]  
data_inputs = ["Text Data", "Image Data"]  
  
for model, data in zip(models, data_inputs):  
    execute_prediction(model, data)
```

3.1.3 魔法方法 (Magic Methods) 与对象模型

魔法方法是 Python 类中以双下划线开头和结尾的特殊方法。它们定义了对象在特定操作（如算术运算、字符串表示、索引访问）下的行为。

1. `__str__` 与 `__repr__`

`__str__`：定义对象被 `str()` 函数转换或 `print()` 打印时的字符串表示，面向用户，强调可读性。

`__repr__`：定义对象被 `repr()` 函数转换时的字符串表示，面向开发者，强调准确性。

```
class Vector:

    def __init__(self, x, y):

        self.x = x

        self.y = y

    def __str__(self):

        return f"Vector({self.x}, {self.y})"

v = Vector(10, 20)

print(v) # 输出: Vector(10, 20), 自动调用了 v.__str__()
```

2. `__call__`

`__call__` 方法允许实例像函数一样被调用。这是 PyTorch 等 AI 框架的核心机制，使得模型对象可以直接接受输入并返回输出。

```
class LinearLayer:

    def __init__(self, k, b):

        self.k = k

        self.b = b

    def __call__(self, x):

        return self.k * x + self.b

layer = LinearLayer(2, 1) # 实例化

result = layer(5)          # 实例被直接调用，等同于 layer.__call__(5)

print(result) # 输出: 11
```

3. 其他常用魔法方法

`__len__(self)`: 定义对对象执行 `len()` 时的行为（常用于定义数据集大小）。

`__getitem__(self, index)`: 定义对象通过索引 `obj[index]` 访问时的行为（常用于获取数据集中的样本）。

3.1.4 迭代器与生成器 (`__iter__`, `yield`)

在处理大规模数据集时，将所有数据一次性加载到内存是不可行的。Python 提供了迭代器协议和生成器机制来实现数据的流式处理（惰性求值）。

1. 迭代器协议 (Iterator Protocol)

可迭代对象 (Iterable): 实现了 `__iter__` 方法的对象，可以被 `for` 循环遍历。

迭代器 (Iterator): 实现了 `__iter__` 和 `__next__` 方法的对象，负责维护遍历的状态。

2. 生成器 (Generator) 与 `yield`

生成器是一种简化的迭代器实现方式。当函数中包含 `yield` 关键字时，该函数变为生成器函数。

调用生成器函数返回一个生成器对象。每次调用 `next()` 或在循环中遍历时，代码执行到 `yield` 语句会暂停，并返回当前值；下次调用时，从暂停处继续执行。

传统方式：构建完整列表（占用 $O(N)$ 内存）

```
def get_squares_list(n):  
    result = []  
    for i in range(n):  
        result.append(i ** 2)  
    return result
```

生成器方式：按需生成数据（占用 $O(1)$ 内存）

```
def get_squares_gen(n):  
    for i in range(n):  
        yield i ** 2 # 返回数据并暂停执行
```

使用生成器

```
gen = get_squares_gen(5)  
# print(gen) # 输出生成器对象信息
```

```
for num in gen:  
    print(num) # 依次输出 0, 1, 4, 9, 16
```

3. 在 AI 数据处理中的应用

在深度学习训练中，生成器用于构建数据管道（Data Pipeline）：

- （1）程序请求一个批次（Batch）的数据。
- （2）生成器从磁盘读取文件。
- （3）对数据进行预处理。
- （4）yield 返回处理后的数据供模型训练。
- （5）释放内存，准备处理下一批次。

这种机制确保了无论数据集总量多大，内存中仅存在当前批次的数据，从而支持大规模数据的训练。

3.2 异常处理

在构建复杂的软件系统时，错误处理的能力决定了系统的健壮性。Python 的异常机制不仅仅用于防止程序崩溃，它更是一种控制程序流程、传递错误信息和管理资源的结构化方式。

3.2.1 异常传播机制与 Traceback 分析

1. 异常传播（Exception Propagation）

当一个函数内部引发异常且未被捕获时，该异常不会立刻导致程序崩溃，而是会向上传播（Propagate）。它会沿着调用栈（Call Stack）一层一层地向外抛出，直到遇到一个匹配的 except 块或者到达主程序的最顶层（此时解释器会终止程序并打印错误信息）。

理解调用栈对于调试深度学习框架（如 PyTorch/TensorFlow）的代码尤为重要，因为错误往往发生在框架深层的函数中。

```
def layer_3():  
    print(" 进入 Layer 3")  
    # 抛出一个 ValueError  
    raise ValueError("数据维度不匹配")  
    print(" Layer 3 结束") # 这行代码不会执行  
  
def layer_2():  
    print(" 进入 Layer 2")
```

```
layer_3() # 调用 layer_3

print(" Layer 2 结束")

def layer_1():
    print("进入 Layer 1")
    try:
        layer_2() # 调用 layer_2
    except ValueError as e:
        print(f'在 Layer 1 捕获异常: {e}')
```

执行流程

```
layer_1()
```

输出结果:

进入 Layer 1

进入 Layer 2

进入 Layer 3

在 Layer 1 捕获异常: 数据维度不匹配

说明: 异常从 layer_3 产生, 传播到 layer_2 (未捕获), 继续传播到 layer_1, 最终被捕获。layer_3 和 layer_2 在异常点之后的代码均未执行。

2. Traceback 分析

当异常未被捕获时, Python 会打印 Traceback (回溯) 信息。Traceback 按执行顺序倒序记录了异常发生时的函数调用路径。

典型的 Traceback 结构如下:

Traceback (most recent call last):

```
File "main.py", line 15, in <module>
```

```
    layer_1()
```

```
File "main.py", line 11, in layer_1
```

```
    layer_2()
```

```
File "main.py", line 7, in layer_2
```

```
    layer_3()
```

```
File "main.py", line 4, in layer_3
```

```
    raise ValueError("数据维度不匹配")
```

```
ValueError: 数据维度不匹配
```

- (1) **Most recent call last:** 提示最后一次调用在最下方。
- (2) **调用链:** 从上到下展示了 `layer_1 -> layer_2 -> layer_3` 的调用过程。
- (3) **错误位置:** 最后一行指出了具体的错误类型和错误信息，以及引发错误的具体代码行。

3.2.2 自定义异常类 (User-defined Exceptions)

虽然 Python 内置了丰富的异常类（如 `ValueError`, `TypeError`），但在特定领域的应用中，使用通用的异常往往无法清晰表达错误的语义。通过自定义异常类，我们可以创建特定于业务逻辑的错误类型。

实现方式

自定义异常类通常继承自 Python 的内置 `Exception` 类。

```
class ALError(Exception):
```

```
    """AI 项目的基础异常类"""
```

```
    pass
```

```
class DimensionMismatchError(ALError):
```

```
    """当张量或矩阵维度不匹配时抛出"""
```

```
    def __init__(self, expected, actual):
```

```
        self.expected = expected
```

```
        self.actual = actual
```

```
        # 调用父类的初始化方法，设置错误信息
```

```
        super().__init__(f'维度不匹配: 期望 {expected}, 实际 {actual}')
```

```
class ModelNotConvergedError(ALError):
```

```
"""当模型训练未收敛时抛出"""

def __init__(self, epoch, loss):
    super().__init__(f"训练在 Epoch {epoch} 停止, Loss 为 {loss} (未达
标)")

# 使用自定义异常

def matrix_multiply(a_shape, b_shape):
    if a_shape[1] != b_shape[0]:
        # 抛出具有明确语义的异常
        raise DimensionMismatchError(expected=a_shape[1], actual=b_shape[0])

    print("矩阵乘法运算...")

try:
    matrix_multiply((10, 5), (4, 20))
except DimensionMismatchError as e:
    # 可以通过属性访问详细的错误数据, 便于日志记录或自动修复
    print(f"错误捕获: {e}")
    print(f"Debug Info - 期望列数: {e.expected}, 实际行数: {e.actual}")
```

这种做法使得上层调用者可以针对性地捕获 `DimensionMismatchError` 并进行处理, 而不是捕获所有 `ValueError`。

3.2.3 with 上下文管理器与资源安全

1. 资源管理的问题

在程序中, 涉及到文件 I/O、数据库连接、线程锁或网络套接字时, 必须确保资源在使用后被正确释放 (关闭)。如果仅依赖 `try...finally` 结构, 代码会变得冗长且容易出错。

2. 上下文管理器协议 (Context Manager Protocol)

Python 提供了 `with` 语句来简化资源管理。任何实现了 `__enter__` 和 `__exit__` 方法的类, 都可以作为一个上下文管理器使用。

(1) `__enter__(self)`: 进入 `with` 语句块时调用, 返回资源对象。

(2) `__exit__(self, exc_type, exc_val, exc_tb)`: 离开 `with` 语句块时自动调用（无论是否发生异常）。

3. 实战：自定义计时器

在 AI 训练中，我们经常需要统计某个模块的耗时。

```
import time

class Timer:

    def __enter__(self):
        self.start = time.time()

        return self # 返回对象本身

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.end = time.time()

        self.interval = self.end - self.start

        print(f'代码块执行耗时: {self.interval:.4f} 秒')

        # 如果返回 False (默认), 异常会继续向外传播
        # 如果返回 True, 异常会被这里的 __exit__ 吞掉 (不再抛出)

        return False

# 使用自定义上下文管理器
with Timer() as t:

    # 模拟耗时操作

    result = sum(i**2 for i in range(1000000))

print("程序继续执行...")
```

4. contextlib 模块

Python 标准库 `contextlib` 提供了装饰器 `@contextmanager`，允许通过生成器函数快速定义上下文管理器，无需编写类。

```
from contextlib import contextmanager
```

```
@contextmanager
def temp_resource():
    print("资源初始化...")
    yield "Resource"
    print("资源清理...")

with temp_resource() as res:
    print(f'正在使用 {res}')
```

3.2.4 编写健壮代码的最佳实践

1. EAFP 原则 (Easier to Ask for Forgiveness than Permission)

Python 社区推崇 EAFP 风格，即“先执行代码，出了错再处理”，而不是 LBYL (Look Before You Leap，先检查再执行)。

(1) LBYL (C 语言风格):

```
if "key" in my_dict:
    value = my_dict["key"]
else:
    # 处理不存在的情况
```

(2) EAFP (Python 风格):

```
try:
    value = my_dict["key"]
except KeyError:
    # 处理不存在的情况
```

EAFP 风格通常更高效，因为在大多数正常情况下，它少了一次条件判断。

2. 捕获具体的异常

永远不要使用裸露的 `except:` 或 `except Exception:`，除非是在程序的最外层进行兜底日志记录。

(1) 错误做法:

```
try:
    process_data()
except: # 捕获了包括 KeyboardInterrupt (Ctrl+C) 在内的所有异常
    pass # 错误被静默吞没，调试极其困难
```

(2) 正确做法:

```
try:
    process_data()
except (ValueError, IndexError) as e:
    logging.error(f'数据处理失败: {e}')
```

3. 保持 Traceback 完整

在捕获异常并重新抛出时，应直接使用 `raise` 而不是 `raise e`，以保留原始的调用栈信息。

```
try:
    complex_calculation()
except ArithmeticError as e:
    logging.warning("计算出错，正在重试...")
    # 若重试失败需要抛出异常
    raise # 这里的 raise 会保留 complex_calculation 的原始报错位置
```

通过遵循这些原则，可以编写出既稳定又易于维护的 Python 代码，从容应对 AI 系统中可能出现的各种边缘情况。

3.3 正则表达式

正则表达式使用单个字符串来描述、匹配一系列符合某个句法规则的字符串。Python 通过标准库 `re` 模块提供了对正则表达式的完整支持。

3.3.1 正则表达式基础语法与元字符

正则表达式的强大之处在于使用元字符 (Meta-characters)。这些字符在正则表达式引擎中具有特殊的含义，不再代表字符本身。

1. 原始字符串 (Raw String)

在 Python 中编写正则表达式时，建议始终使用 `r` 前缀（例如 `r"\d+"`）。

（1）普通字符串中，`\` 是转义符（例如 `\n` 是换行）。

（2）原始字符串中，`\` 被视为普通字符。这避免了正则表达式中的反斜杠与 Python 字符串转义发生冲突。

2. 常用元字符分类

（1）字符匹配

`.`: 匹配除换行符 `\n` 之外的任意单个字符。

`\d`: 匹配任意数字，等价于 `[0-9]`。

`\D`: 匹配任意非数字。

`\w`: 匹配字母、数字、下划线（单词字符），等价于 `[a-zA-Z0-9_]`。

`\W`: 匹配非单词字符。

`\s`: 匹配任意空白字符（空格、制表符、换行符）。

`[]`: 字符集合，匹配括号内的任意一个字符。例如 `[aeiou]` 匹配任意元音字母。

（2）数量修饰 (Quantifiers)

`*`: 重复零次或多次。

`+`: 重复一次或多次。

`?`: 重复零次或一次。

`{n}`: 重复确定的 `n` 次。

`{n,m}`: 重复 `n` 到 `m` 次。

（3）边界匹配 (Anchors)

`^`: 匹配字符串的开头。

`$`: 匹配字符串的结尾。

（4）逻辑与分组

`|`: 逻辑或。例如 `a|b` 匹配 `'a'` 或 `'b'`。

`()`: 分组。将多个字符视为一个整体，并保存匹配的子组。

示例代码：

```
import re
```

```
text = "Error code: 404, Response time: 20ms"
```

```
# 匹配所有连续的数字
```

```
pattern = r"\d+"
```

```
# 解释: \d 表示数字, + 表示出现一次或多次
```

3.3.2 re 模块核心函数详解

re 模块提供了多种函数来处理不同的匹配需求。理解 `match`、`search` 和 `findall` 的区别是关键。

1. `re.match(pattern, string)`

功能: 尝试从字符串的起始位置匹配模式。

返回: 如果起始位置匹配成功, 返回一个匹配对象 (Match Object); 否则返回 `None`。

```
text = "Python is powerful"
```

```
# 匹配成功
```

```
result = re.match(r"Python", text)
```

```
print(result.group()) # 输出: Python
```

```
# 匹配失败 (因为起始位置不是 "powerful")
```

```
result = re.match(r"powerful", text)
```

```
print(result) # 输出: None
```

2. `re.search(pattern, string)`

功能: 扫描整个字符串, 返回第一个成功的匹配。

返回: 匹配对象或 `None`。这是最常用的查找函数。

```
text = "Error: Connection timeout at 10:00 AM"
```

```
# 查找 "Connection"
```

```
result = re.search(r"Connection", text)
```

```
if result:
```

```
    print(f'找到匹配, 位置: {result.span()}')
```

```
    # 输出: 找到匹配, 位置: (7, 17)
```

3. `re.findall(pattern, string)`

功能: 扫描整个字符串，找到所有匹配的子串。

返回: 一个列表 (List)。如果没有匹配，返回空列表。

```
text = "IP addresses: 192.168.1.1 and 10.0.0.1"

# 匹配 IP 地址的简单模式（数字.数字.数字.数字）

ips = re.findall(r"\d+\.\d+\.\d+\.\d+", text)

print(ips)

# 输出: ['192.168.1.1', '10.0.0.1']
```

4. re.sub(pattern, repl, string)

功能: 搜索并替换。将匹配到的部分替换为指定的字符串 `repl`。

场景: 数据清洗，去除噪声。

```
text = "Contact:   user@example.com   "

# 去除所有空白字符（替换为空字符串）

clean_text = re.sub(r"\s+", "", text)

print(clean_text)

# 输出: Contact:user@example.com
```

5. re.compile(pattern)

功能: 将正则表达式编译成一个 `Pattern` 对象。

优势: 如果一个模式需要被重复使用多次（例如在循环中处理数万行日志），先编译可以显著提高执行效率。

```
# 预编译模式

email_pattern = re.compile(r"[\w\.-]+@[\w\.-]+")

users = ["user1@gmail.com", "admin@site.org", "invalid-email"]

for user in users:

    if email_pattern.match(user):

        print(f'有效邮箱: {user}')
```

3.3.3 实战：从非结构化文本中提取关键信息

在构建 AI 数据集时，源数据通常是非结构化的日志或网页文本。我们需要将其转换为结构化的字典或表格。

(1) 场景描述:

服务器日志包含时间戳、日志级别和具体消息。我们需要解析这些日志。

(2) 原始数据:

2023-10-01 08:30:00 [INFO] System started.

2023-10-01 08:31:05 [ERROR] Connection failed: timeout.

(3) 解析代码:

```
import re

log_data = """
2023-10-01 08:30:00 [INFO] System started.
2023-10-01 08:31:05 [ERROR] Connection failed: timeout.
"""

# 定义正则表达式
# 使用圆括号 () 创建分组，以便单独提取时间、级别和消息
# (?P<name>...) 是命名分组语法，可以直接通过名称获取内容
log_pattern = re.compile(r"(?P<time>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})
\[ (?P<level>\w+)\] (?P<msg>.*)")

# 按行处理
for line in log_data.strip().split("\n"):
    match = log_pattern.match(line)
    if match:
        # groupdict() 将命名分组直接转换为字典
        data = match.groupdict()
        print(f'时间: {data['time']}, 级别: {data['level']}, 消息: {data['msg']}")

# 输出:
# 时间: 2023-10-01 08:30:00, 级别: INFO, 消息: System started.
```

时间: 2023-10-01 08:31:05, 级别: ERROR, 消息: Connection failed: timeout.

通过这种方式，原本杂乱的字符串被转化为了结构化的数据，可以直接存入数据库或 Pandas DataFrame 进行后续分析。这是数据工程链路中至关重要的一环。

3.4 并行编程：

3.4.1 多线程 (Threading)

3.4.2 多进程 (Multiprocessing)

3.4.2 进程间通信

3.5 设计模式：

概要说明:: 创建型模式、结构型模式、行为型模式。这些模式可以解决代码复用、解耦、扩展性差等常见问题，也是面试和实际开发中的高频知识点。

介绍：

3.5.1 装饰器模式：

Python 原生支持，日志、权限、缓存等场景必备。

3.5.2 单例模式：

数据库连接、配置管理等唯一实例场景。

3.5.3 工厂模式：

对象创建逻辑复杂时的解耦利器。

3.5.4 观察者模式：

事件驱动、消息通知系统的核心。

3.5.5 策略模式：

多算法动态切换的最佳实践

3.6 网络编程

网络编程是指编写运行在多个设备（计算机）上的程序，这些设备通过网络连接进行通信。Python 提供了从底层网络接口（Socket）到应用层协议（HTTP）的全套库支持。

3.6.1 网络通讯基础：TCP/IP 与 UDP 协议概览

网络通信依赖于一系列标准协议。在传输层，最核心的两个协议是 TCP 和 UDP。

1. TCP (Transmission Control Protocol, 传输控制协议)

TCP 是一种面向连接的、可靠的、基于字节流的传输层通信协议。

- (1) 面向连接：通信双方必须先建立连接（三次握手）才能传输数据。
- (2) 可靠性：TCP 通过序列号、确认应答、重发控制等机制，确保数据无丢失、无重复、按序到达。
- (3) 应用场景：绝大多数互联网应用，如 HTTP (网页/API)、FTP (文件传输)、SMTP (邮件)。AI 领域的数据下载和模型推理接口主要基于 TCP。

2. UDP (User Datagram Protocol, 用户数据报协议)

UDP 是一种无连接的、不可靠的传输协议。

- (1) 无连接：发送数据前不需要建立连接，直接发送。
- (2) 不可靠：不保证数据送达，也不保证顺序，但传输效率高，延迟低。
- (3) 应用场景：实时性要求高但允许少量丢包的场景，如视频流媒体、在线游戏、实时传感器数据采集。

3. 协议对比

特性	TCP	UDP
连接性	面向连接	无连接
可靠性	高（保证交付、顺序）	低（尽最大努力交付）
速度	较慢（由于握手和确认机制）	快
数据模式	字节流	数据报文

3.6.2 Socket 编程基础

Socket（套接字）是网络编程的一个抽象概念，它是网络通信的端点。Python 的 socket 模块提供了访问 BSD Socket 接口的途径，允许开发者直接控制底层的 TCP/UDP 连接。

1. TCP 通信流程：

服务端：创建 Socket -> 绑定 IP 和端口 (Bind) -> 监听 (Listen) -> 接受连接 (Accept) -> 接收/发送数据 -> 关闭。

客户端：创建 Socket -> 连接服务端 (Connect) -> 发送/接收数据 -> 关闭。

2. 代码示例：实现一个简单的 TCP Echo Server（回声服务器）

（1）服务端代码 (server.py)：

```
import socket

# 1. 创建 socket 对象
# AF_INET 表示 IPv4 地址族，SOCK_STREAM 表示 TCP 协议
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# 2. 绑定 IP 地址和端口
# '0.0.0.0' 表示监听本机所有网络接口，端口设为 8888
server_socket.bind(('0.0.0.0', 8888))

# 3. 开始监听，参数 5 表示挂起的连接队列最大长度
server_socket.listen(5)
print("服务器启动，等待连接...")

while True:
    # 4. 接受客户端连接
    # accept() 会阻塞，直到有客户端连接。返回一个新的 socket 和客户端地址
    client_sock, addr = server_socket.accept()
    print(f"接受连接来自: {addr}")
```

5. 接收数据 (缓冲区大小 1024 字节)

```
data = client_sock.recv(1024)
```

if data:

```
print(f'收到消息: {data.decode('utf-8')}')
```

6. 发送数据回客户端

```
client_sock.send(f'服务器已收到: {data.decode('utf-8')}'.encode('utf-8'))
```

7. 关闭与该客户端的连接

```
client_sock.close()
```

(2) 客户端代码 (client.py):

```
import socket
```

1. 创建 socket 对象

```
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

2. 连接服务器 (IP 地址需根据实际情况修改, 本地测试用 127.0.0.1)

```
client_socket.connect(('127.0.0.1', 8888))
```

3. 发送数据

```
message = "Hello, AI Network!"
```

```
client_socket.send(message.encode('utf-8'))
```

4. 接收响应

```
response = client_socket.recv(1024)
```

```
print(f'来自服务器的响应: {response.decode('utf-8')}')
```

5. 关闭连接

```
client_socket.close()
```

3.6.3 Python 实现 HTTP 请求 (requests / urllib)

在实际的 AI 开发中，直接使用 Socket 较为繁琐。我们更多地工作应用层，即 HTTP 协议。Python 拥有强大的 HTTP 客户端库。

1. urllib (标准库)

Python 内置库，功能完整但 API 设计较为底层，使用起来较为冗长。

2. requests (第三方库)

Python 社区的事实标准，被称为“给人类使用的 HTTP 库”。它封装了复杂的底层逻辑，提供了极简的 API。

安装：pip install requests

3. 常用操作示例：

(1) GET 请求 (获取数据)

常用于获取网页内容或查询 API。

```
import requests

# 目标 URL (GitHub 公开 API)
url = "https://api.github.com/events"

# 发送 GET 请求
response = requests.get(url)

# 检查状态码 (200 表示成功)
if response.status_code == 200:
    print("请求成功")
    # 获取响应内容
    # response.text 返回字符串
    # response.json() 自动将 JSON 响应解析为 Python 字典/列表
    data = response.json()
    print(f'获取到 {len(data)} 条事件数据')
else:
```

```
print(f'请求失败: {response.status_code}')
```

(2) POST 请求 (提交数据)

常用于向 AI 模型服务发送推理数据。

```
import requests

import json

url = "http://api.example.com/predict"

# 构造请求数据

payload = {"input_text": "人工智能发展迅速", "model": "v1"}

# 发送 POST 请求, 自动处理 Content-Type 和 JSON 序列化

response = requests.post(url, json=payload)

if response.status_code == 200:

    result = response.json()

    print(f'推理结果: {result}')
```

3.6.4 网络爬虫实战：获取 AI 训练数据

网络爬虫（Web Crawler）是一种自动浏览万维网并抓取信息的程序。对于 AI 训练而言，互联网是最大的非结构化数据源。

1. 基本流程：

- （1）发送请求：使用 requests 获取网页 HTML 源码。
- （2）解析内容：使用解析库（如 BeautifulSoup 或 lxml）提取目标数据。
- （3）数据清洗：使用正则表达式或字符串操作清洗数据。
- （4）持久化存储：将数据保存到文件或数据库。

2. 实战代码：抓取某科技新闻网站的标题

前置依赖：需要安装 beautifulsoup4 库 (pip install beautifulsoup4)。

```
import requests

from bs4 import BeautifulSoup
```

```
import time

def fetch_headlines(url):
    # 设置 User-Agent 伪装成浏览器，防止被简单的反爬虫机制拦截
    headers = {
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36"
    }

    try:
        # 1. 发送请求
        response = requests.get(url, headers=headers, timeout=10)
        response.raise_for_status() # 如果状态码不是 200，抛出异常

        # 2. 解析内容
        soup = BeautifulSoup(response.text, "html.parser")

        # 假设新闻标题都在 <h2> 标签的 class="news-title" 中
        # 注意：实际开发中需根据目标网页结构调整选择器
        titles = soup.find_all("h2", class_="news-title")

        extracted_data = []
        for title in titles:
            # get_text(strip=True) 获取标签内的纯文本并去除首尾空白
            text = title.get_text(strip=True)
            if text:
                extracted_data.append(text)

        return extracted_data
```

```
except requests.RequestException as e:

    print(f"请求发生错误: {e}")

    return []

# 调用示例 (使用伪代码 URL)

target_url = "https://news.ycombinator.com/"

# 注意: 对真实网站爬取需遵守 robots.txt 协议

headlines = fetch_headlines(target_url)

print(f"抓取到 {len(headlines)} 条标题:")

for i, title in enumerate(headlines[:5]):

    print(f"{i+1}. {title}")
```

3. 注意事项:

- (1) **Robots 协议:** 在爬取网站前, 应检查其 robots.txt 文件, 确认哪些页面允许爬取。
- (2) **请求频率:** 应控制请求频率 (如使用 `time.sleep`), 避免对目标服务器造成过大负载 (DDoS 攻击)。
- (3) **合法性:** 尊重版权和隐私数据, 仅抓取公开数据用于学习和研究。

3.7 数据库交互

数据库 (Database) 是按照数据结构来组织、存储和管理数据的仓库。与普通文本文件相比, 数据库提供了更高效的数据检索、更新机制以及数据完整性保障。

3.7.1 数据库基础概念与 SQLite 简介

1. 关系型数据库 (RDBMS)

关系型数据库基于关系模型, 使用表 (Table) 来存储数据。

- (1) **表 (Table):** 由行和列组成的二维结构。
- (2) **列 (Column):** 定义数据的字段 (属性) 和类型 (如整数、文本)。

-
- (3) 行 (Row): 表中的一条具体记录。
 - (4) 主键 (Primary Key): 唯一标识表中每一行的列, 确保记录不重复。
 - (5) SQL (Structured Query Language): 结构化查询语言, 用于管理和操作数据库的标准语言。

2. SQLite

SQLite 是一个轻量级、基于文件的关系型数据库引擎。

- (1) 无服务器: 不需要配置和启动独立的数据库服务器进程, 直接读写本地文件。
- (2) 零配置: Python 标准库内置了 `sqlite3` 模块, 无需安装额外驱动即可使用。
- (3) 应用场景: 嵌入式设备、移动应用、以及 AI 项目的原型开发和中小规模数据集存储。对于大规模并发或海量数据, 通常会迁移到 MySQL 或 PostgreSQL。

3.7.2 使用 Python 操作数据库 (CRUD 增删改查)

Python 操作数据库遵循 DB-API 2.0 标准规范。操作流程通常包括: 建立连接、创建游标、执行 SQL、提交事务、关闭连接。

1. 核心步骤:

- (1) 连接 (Connection): 建立与数据库文件的通道。
- (2) 游标 (Cursor): 用于执行 SQL 语句并获取结果的对象。
- (3) 事务 (Transaction): 一系列操作的集合。更改数据 (增删改) 后需要提交 (commit) 才能生效。

2. CRUD 操作代码示例

```
import sqlite3

# 1. 连接数据库
# 如果文件不存在, 会自动在当前目录下创建 'ai_data.db'
conn = sqlite3.connect('ai_data.db')

# 2. 创建游标对象
cursor = conn.cursor()
```

3. 创建表 (Create)

使用 SQL 语句定义表结构: id (主键), title (文本), score (浮点数)

```
create_table_sql = ""
```

```
CREATE TABLE IF NOT EXISTS headlines (
```

```
    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    title TEXT NOT NULL,
```

```
    score REAL
```

```
);
```

```
""
```

```
cursor.execute(create_table_sql)
```

4. 插入数据 (Insert)

注意: 使用 ? 作为占位符, 避免 SQL 注入风险

```
insert_sql = "INSERT INTO headlines (title, score) VALUES (?, ?)"
```

```
data = ("AI 模型准确率突破 99%", 0.95)
```

```
cursor.execute(insert_sql, data)
```

批量插入

```
data_list = [
```

```
    ("Python 发布新版本", 0.88),
```

```
    ("深度学习入门教程", 0.92)
```

```
]
```

```
cursor.executemany(insert_sql, data_list)
```

提交事务 (必须执行, 否则数据不会写入文件)

```
conn.commit()
```

5. 查询数据 (Read)

```
select_sql = "SELECT * FROM headlines WHERE score > 0.9"
```

```
cursor.execute(select_sql)

# 获取查询结果的所有行
results = cursor.fetchall()
print("查询结果:")
for row in results:
    # row 是一个元组: (id, title, score)
    print(row)

# 6. 更新数据 (Update)
update_sql = "UPDATE headlines SET score = ? WHERE title = ?"
cursor.execute(update_sql, (0.99, "Python 发布新版本"))
conn.commit()

# 7. 删除数据 (Delete)
delete_sql = "DELETE FROM headlines WHERE score < 0.9"
cursor.execute(delete_sql)
conn.commit()

# 8. 关闭游标和连接
cursor.close()
conn.close()
```

3.7.3 实战：将爬取的数据持久化存储

结合 4.6 节的网络爬虫知识，我们将抓取到的新闻标题存储到 SQLite 数据库中，构建一个简单的 AI 训练语料库。

1. 代码实现

```
import sqlite3

import requests
```

```

from bs4 import BeautifulSoup

class NewsScraperDB:

    def __init__(self, db_name="corpus.db"):
        self.conn = sqlite3.connect(db_name)
        self.cursor = self.conn.cursor()
        self._init_table()

    def _init_table(self):
        """初始化数据表"""
        self.cursor.execute("""
            CREATE TABLE IF NOT EXISTS news (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                title TEXT UNIQUE, -- 标题唯一，防止重复抓取
                source_url TEXT,
                fetched_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            )
        """)
        self.conn.commit()

    def save_news(self, title, url):
        """保存单条新闻，忽略重复项"""
        try:
            self.cursor.execute(
                "INSERT OR IGNORE INTO news (title, source_url) VALUES
(?)",
                (title, url)
            )
            self.conn.commit()

```

```
except sqlite3.Error as e:

    print(f'数据库错误: {e}')


def close(self):

    self.cursor.close()

    self.conn.close()


def fetch_and_store(url):

    # 模拟爬虫逻辑 (简化版)

    headers = {"User-Agent": "Mozilla/5.0"}

    try:

        response = requests.get(url, headers=headers, timeout=5)

        soup = BeautifulSoup(response.text, "html.parser")

        # 假设我们爬取 Hacker News

        links = soup.select(".titleline > a")

        db = NewsScraperDB()

        count = 0

        for link in links:

            title = link.get_text()

            href = link.get('href')

            # 存储到数据库

            db.save_news(title, href)

            count += 1

        print(f'成功抓取并存储 {count} 条数据。')
```

```
# 验证存储结果

db.cursor.execute("SELECT count(*) FROM news")

total = db.cursor.fetchone()[0]

print(f'数据库中当前共有 {total} 条记录。')


db.close()
```

```
except Exception as e:

    print(f'发生错误: {e}')
```

```
# 运行爬虫并存储

target_url = "https://news.ycombinator.com/"

fetch_and_store(target_url)
```

2. 关键点解析:

(1) 类封装: 将数据库连接、表初始化、插入操作封装在 `NewsScraperDB` 类中, 提高代码复用性。

(2) `UNIQUE` 约束与 `INSERT OR IGNORE`: 在 SQL 建表时设置 `title` 为唯一, 在插入时使用 `IGNORE` 关键字。这是防止爬虫重复抓取同一条新闻的有效手段。

(3) 持久化价值: 程序运行结束后, 数据依然保存在 `corpus.db` 文件中。后续可以随时编写新的 Python 脚本读取该数据库, 进行分词、情感分析或模型训练。

3.8 图形用户界面 (GUI)

图形用户界面是指采用图形方式显示的计算机操作用户界面。与命令行界面 (CLI) 不同, GUI 允许用户通过鼠标点击、键盘输入等交互方式与程序进行非线性的沟通。

3.8.1 GUI 编程基本概念 (事件驱动)

传统的脚本程序通常按照从上到下的顺序执行, 执行完毕后程序退出。而 GUI 程序基于事件驱动 (Event-Driven) 架构, 其运行机制具有本质区别。

1. 主事件循环 (Main Event Loop)

GUI 程序启动后，会进入一个无限循环（通常称为 `mainloop`）。在这个循环中，程序持续监听来自操作系统的事件信号。程序不会主动退出，除非接收到关闭信号。

2. 事件 (Event) 与 绑定 (Binding)

事件：用户的每一个操作都被封装为一个“事件”对象。例如：鼠标左键点击 (`<Button-1>`)、键盘按下 (`<KeyPress>`)、窗口大小改变 (`<Configure>`)。

绑定：将特定的“事件”与特定的“函数”关联起来的过程。

3. 回调函数 (Callback Function)

当绑定的事件发生时，主循环会调用预先定义好的函数来响应该事件。这个函数被称为回调函数。

执行流程示意：

启动程序 -> 构建界面 -> 进入主循环 (等待) -> 用户点击按钮 -> 触发点击事件 -> 执行回调函数 (处理逻辑) -> 更新界面 -> 返回主循环 (继续等待)。

3.8.2 常用控件与布局管理 (基于 Tkinter)

Python 内置的 `tkinter` 模块是 Tcl/Tk GUI 工具包的 Python 接口。它无需安装第三方库，且足以应对大多数中小型 AI 工具的开发需求。

1. 常用控件 (Widgets)

控件是构成 GUI 的基本元素。

- (1) Tk: 根窗口（主窗口）。
- (2) Label: 用于显示文本或图像，通常用于展示静态信息或模型输出结果。
- (3) Button: 按钮，用于触发事件（如“开始推理”）。
- (4) Entry: 单行文本输入框，用于获取用户输入（如超参数设置）。
- (5) Text: 多行文本输入框，用于处理长文本（如 NLP 任务的输入）。
- (6) Frame: 容器控件，用于将界面划分为不同的区域。

2. 布局管理 (Geometry Managers)

布局管理器决定了控件在窗口中的位置和大小。`tkinter` 提供了三种主要的布局管理器：

(1) `pack()`: 按添加顺序将控件排列在父容器的顶部、底部、左侧或右侧。适用于简单的垂直或水平堆叠。

(2) `grid()`: 将父容器划分为二维表格（行/列），将控件放置在指定的单元格中。这是最常用的布局方式，适合表单和复杂界面。

(3) `place()`: 使用绝对坐标（`x, y`）或相对坐标定位。灵活性最高，但难以适应窗口大小的改变，不推荐用于主布局。

代码示例：基础布局

```
import tkinter as tk

# 1. 创建主窗口
root = tk.Tk()
root.title("AI 配置面板")
root.geometry("300x200")

# 2. 创建控件
label_lr = tk.Label(root, text="学习率 (Learning Rate):")
entry_lr = tk.Entry(root)
btn_confirm = tk.Button(root, text="保存配置")

# 3. 使用 Grid 布局
# sticky="w" 表示西(West)，即左对齐
# padx/pady 表示水平/垂直外边距
label_lr.grid(row=0, column=0, padx=10, pady=10, sticky="w")
entry_lr.grid(row=0, column=1, padx=10, pady=10)
btn_confirm.grid(row=1, column=0, columnspan=2, pady=20) # columnspan 合并两列

# 4. 进入主循环
root.mainloop()
```

3.8.3 实战：为 AI 模型制作可视化交互界面

本节我们将结合同面向对象编程（OOP）的思想，构建一个“情感分析演示系统”。用户输入一段文本，点击按钮后，系统模拟 AI 模型进行推理，并显示情感倾向（积极/消极）。

代码实现

```
import tkinter as tk

from tkinter import messagebox

import random

import time


class SentimentApp:

    def __init__(self, root):

        self.root = root

        self.root.title("AI 情感分析系统 v1.0")

        self.root.geometry("500x350")

        # 初始化界面组件

        self._setup_ui()

    def _setup_ui(self):

        """构建 GUI 界面"""

        # 顶部标题

        self.header = tk.Label(

            self.root,

            text="文本情感倾向检测",

            font=("Arial", 16, "bold"),

            fg="#333"

        )

        self.header.pack(pady=20)
```

```
# 输入区域容器

input_frame = tk.Frame(self.root)

input_frame.pack(pady=10)

tk.Label(input_frame, text="请输入待分析文本: ").pack(anchor="w")

# 多行文本输入框

self.text_input = tk.Text(input_frame, height=5, width=50)

self.text_input.pack()

# 操作按钮

self.btn_analyze = tk.Button(

    self.root,

    text="开始分析",

    command=self.run_inference, # 绑定事件处理函数

    bg="#007bff",

    fg="white",

    font=("Arial", 12),

    width=20

)

self.btn_analyze.pack(pady=15)

# 结果显示区域

self.result_label = tk.Label(

    self.root,

    text="等待分析...",

    font=("Arial", 14),

    fg="gray"

)
```

```
self.result_label.pack(pady=10)

def mock_ai_model(self, text):
    """模拟耗时的 AI 推理过程"""
    time.sleep(1.0) # 模拟计算延迟
    # 这里使用随机逻辑代替真实模型
    # 在实际项目中，这里会调用 PyTorch/TensorFlow 模型
    score = random.random()

    if "差" in text or "烂" in text or "失败" in text:
        return 0.1 # 强制负面

    return score

def run_inference(self):
    """按钮点击的回调函数"""

    # 1. 获取输入 (从第 1 行第 0 列字符开始，读到末尾)
    # "end-1c" 是为了去除 Text 控件自动添加的末尾换行符
    user_text = self.text_input.get("1.0", "end-1c").strip()

    if not user_text:
        messagebox.showwarning("提示", "请输入文本内容！")
        return

    # 2. 更新界面状态
    self.result_label.config(text="正在推理中...", fg="blue")
    self.root.update() # 强制刷新界面，否则在 sleep 时界面会卡死

    # 3. 调用 AI 模型
    try:
        score = self.mock_ai_model(user_text)
```

```
# 4. 根据结果更新界面

if score > 0.6:

    result_text = f"积极 (置信度: {score:.2f})"

    color = "green"

elif score < 0.4:

    result_text = f"消极 (置信度: {1-score:.2f})"

    color = "red"

else:

    result_text = f"中性 (置信度: {score:.2f})"

    color = "#d9534f" # 橙色

self.result_label.config(text=result_text, fg=color)

except Exception as e:

    self.result_label.config(text="推理出错", fg="red")

    messagebox.showerror("错误", str(e))

if __name__ == "__main__":

    # 创建根窗口

    root_window = tk.Tk()

    # 实例化应用逻辑

    app = SentimentApp(root_window)

    # 启动主循环

    root_window.mainloop()
```

关键技术点解析：

(1) 类封装 (`class SentimentApp`): 将界面组件 (`self.text_input`, `self.result_label`) 作为实例属性, 确保在不同的方法 (如 `run_inference`) 中都能访问和修改这些控件。

(2) 数据获取: `Text` 控件的内容获取需要使用特定的索引语法 `get("1.0", "end-1c")`, 表示从第 1 行第 0 个字符开始, 一直读取到倒数第一个字符 (去除系统自动添加的换行)。

(3) 界面刷新: 在执行耗时操作 (如模型推理) 前, 调用 `self.root.update()` 是必要的, 否则用户在等待计算结果时, 界面上的“正在推理中...”提示可能无法及时显示, 因为主循环被计算任务阻塞了。

通过这种方式, 我们成功将一个后台的逻辑函数封装成了一个普通用户可用的桌面软件。